Exploring Data Memory-Dependent Prefetcher Exploitation on Apple Silicon

Midpoint Report

Mazen Hassouna
Tamim Hasan
Samien Rahman
Humza Ahmad
Nitin Jaladanki
The University of Texas at Austin
USA

Abstract

We investigate the security implications of Apple's data memory-dependent prefetcher (DMP), beginning with the M1. Our scope has broadened from reproducing GoFetch to studying DMP exploitation through both Augury and GoFetch. At the midpoint, our experimental work has focused on Augury activation and reverse engineering. We describe our replication methodology, summarize the available timing outputs and their limitations, and outline the remaining exploitation demonstrations. Our planned novel investigation compares selected activation conditions under Asahi Linux on M1 and on an M4 chip, with particular attention to distinguishing changed behavior from evidence of mitigation.

1 Research Problem and Progress

Our research question is how Apple's DMP can turn memory contents into observable cache activity, and how consistently that behavior appears across access patterns, operating systems, and processor generations. Conventional constant-time programming avoids secret-dependent branches and memory addresses. A DMP complicates this model because hardware can interpret program data as addresses and fetch the corresponding locations even when software does not explicitly dereference those values.

Our original proposal focused on reproducing GoFetch and exploring additional prefetcher side channels on the M1. We have refined this scope to examine M1 DMP exploitation more broadly, using Augury and GoFetch as complementary foundations [1, 2]. Augury supplies the initial array-of-pointers (AoP) characterization and pointer-leakage primitives; GoFetch supplies a refined activation model and demonstrations against constant-time software.

At this stage, we have run experiments aimed at validating Augury's DMP activation criteria and reverse engineering its behavior. The preliminary results below summarize the available timing measurements; we do not present quantitative activation thresholds or reverse-engineered parameters in this report. GoFetch activation, end-to-end exploit demonstrations, and the Asahi Linux and M4 investigations remain future work. We do not yet claim a new attack or a demonstrated defense.

2 Background and Experimental Scope

Augury identifies an AoP prefetcher on Apple processors: a stream of pointer-array accesses can cause hardware to fetch locations referenced by future entries. This creates opportunities to expose pointer values through cache state, including values beyond the bounds of the software's intended array [2]. Its reverse-engineering experiments motivate our tests of training requirements, alignment, stride, prefetch distance, and address boundaries.

GoFetch shows that the activation conditions are broader than the original AoP model suggests. Explicit architectural dereferencing of the candidate pointers is not always necessary. Its refined model includes a history filter, a "do-not-scan" hint associated with cache lines, and restrictions involving the source and target's 4 GB-aligned virtual-address region [1]. These mechanisms make cache residency and experiment ordering important: a negative measurement can reflect suppressed retriggering rather than the absence of a DMP.

Our experiments target controlled local programs and synthetic secrets on the team's hardware. We distinguish detecting DMP-induced cache activity, demonstrating a leakage primitive, and demonstrating cross-process leakage. Success at the first stage does not establish the later stages. macOS on M1 is our replication baseline; Asahi Linux on M1 and the planned M4 tests are comparison environments, not completed results.

3 Approach

3.1 Measurement and Controls

We begin with timing calibration to distinguish cached and uncached accesses. The supplied setup notes report hit measurements around 0–42 ns, misses around 125–167 ns, and a provisional 82 ns classification threshold. These are calibration observations, not universal constants. We will recalibrate for each machine and operating system and retain raw distributions rather than relying only on a threshold. The observed timing granularity does not by itself establish the timer's implementation or its cause.

For each experiment, we will record the chip, operating-system version, compiler options, timer conversion, scheduling configuration, memory layout, and cache-preparation procedure. Repeated runs and randomized trial order will help distinguish a reproducible effect from scheduling noise or residual cache state. Positive controls will establish that the measurement channel detects a known cached target; negative controls will remove or invalidate the relevant pointer pattern while preserving comparable memory traffic.

3.2 Augury Replication

We compare an AoP access pattern with a computed-address baseline and examine whether future, architecturally unaccessed targets become cached. Aggregate speedup is useful supporting evidence, but the target-cache experiment is needed to attribute behavior to pointer-dependent prefetching. We then vary one factor at a time: training length, pointer alignment, stride, lookahead distance, and proximity to the reported 16 KiB and 2 MiB boundary effects. Published values are comparison points, not measurements from our hardware.

After consolidating the activation results, we will run Augury's out-of-bounds pointer-read and address-space layout randomization (ASLR) break demonstrations. Evaluation will report classification accuracy, unsuccessful trials, and the memory-layout assumptions required. We will keep these demonstrations separate from the activation experiments already run.

3.3 GoFetch Replication

We will first reproduce selected activation patterns that distinguish GoFetch's model from Augury's initial characterization: activation without explicit pointer dereferences, processing of in-bounds pointer values, and effects arising from a cache-line fill. We will then vary intervening unique pointers to study history filtering, compare L1-only with L1-and-L2 eviction to test retriggering behavior, and test the 4 GB-aligned source/target restriction.

Once these controls are reliable, we will reproduce a constant-time conditional-swap leakage example with synthetic secrets. We will validate ordinary eviction and observation first, then the compound eviction-set machinery needed for the selected cross-process demonstration. We will report timing distributions and secret-classification accuracy, including false positives. Full cryptographic key extraction remains a stretch goal rather than a required midpoint or final replication result.

3.4 Novel Investigation

Our first direction is to extend the characterization under Asahi Linux on M1. We will port the timing and memory-management portions of the harness, recalibrate them, and repeat selected activation and virtual-address-boundary tests. This asks whether the different operating-system environment exposes behavior that our macOS experiments cannot exercise. We will compare against prior Linux observations before describing any result as novel.

Our second direction is to test both papers' activation patterns on M4. The initial comparison will retain the M1 access patterns while recalibrating timing and cache controls for M4. If a signal disappears, we will check the positive controls, execution placement, cache preparation, and accessible address layouts before attributing the change to mitigation. A negative result establishes only that the tested pattern did not produce detectable leakage under the recorded conditions. If the evidence supports changed or restricted DMP behavior, we will discuss candidate defenses, clearly separating observed behavior from hypotheses about undocumented hardware mechanisms.

4 Preliminary Results and Limitations

The available evidence consists of setup notes and two output files, `baseline.out` and `aop.out`. Each file begins with the same pair of hexadecimal addresses, followed by columns labeled `test` and `train`, and contains 32 measurement rows. Table 1 summarizes these rows without assigning unverified units or meanings to the columns.

The AoP run has a lower median `train` value, but the ranges overlap substantially. All `test` values in both files are below the notes' provisional threshold of 82. If these values are in calibrated nanoseconds, that threshold would classify every `test` observation as a hit in both conditions. Consequently, these logs alone do not demonstrate an AoP-specific hit/miss separation or establish a causal DMP effect. They also do not establish a full speedup curve, prefetch distance, confidence threshold, boundary behavior, or exploit accuracy.

Table 1. Descriptive summary of the supplied logs. Values use the logs' raw units. These are not attack-success measurements.

Statistic	Baseline	AoP
Measurement rows	32	32
`test` range	0–42	0–42
`test` median	41	0
`train` range	417–1250	458–1333
`train` median	1000	812.5

Interpreting these logs causally requires the generating code and configuration, the meaning and units of each column, whether the target was architecturally accessed, and the eviction procedure. We therefore limit our quantitative observations to the descriptive summary above and do not infer activation thresholds, reverse-engineered parameters, or attack success rates from these files.

5 Remaining Tasks and Completion Plan

We will complete the work in the following dependency order, using the existing phased project plan as a guide rather than treating its proposed experiments as completed work.

Consolidate Augury evidence. Document each activation and reverse-engineering experiment with its exact configuration and compare the measured trends against Augury. Repeat ambiguous cases with positive and negative controls. This produces the validated baseline for subsequent comparisons.

Validate GoFetch activation. Run the selected activation patterns and the history, cache-state, and address-region experiments. The completion criterion is reproducible agreement or a documented discrepancy with working controls, not merely successful execution of the artifact.

Run exploit examples. Demonstrate Augury's out-of-bounds read and ASLR break, followed by the GoFetch conditional-swap example. Reserve additional effort for eviction-set construction and cross-process observation. Record success rates and failure cases across repeated runs; separate any single-process demonstration from a cross-process result.

Execute the novel comparisons. Check Asahi Linux setup and M4 availability early, then reuse the validated activation harnesses for both investigations. If resources constrain the work, prioritize the comparison with working measurement infrastructure and preserve a clearly documented boundary on the untested cases. Neither platform study will be described as completed until it has independent controls and repeatable measurements.

Analyze and write. Compare results across configurations, distinguish replication from new observations, and assess potential defenses only to the extent supported by the data. The main risks are scheduling noise, timer resolution, eviction reliability, platform differences, and hardware availability. We will address these through recalibration, staged validation, and early platform checks. The final deliverable will include the report, reproducible experiment instructions, raw measurements, and analysis scripts, with full key extraction optional.

6 Midpoint Assessment

Our project now studies M1 DMP exploitation through both Augury and GoFetch. The completed experimental effort has focused on Augury activation and reverse engineering; the timing logs summarized here provide an initial measurement record rather than a complete quantitative characterization. The remaining work connects that characterization to exploit demonstrations and to controlled Asahi Linux and M4 comparisons. This progression will let us distinguish what we reproduced, what changed across platforms, and what remains unexplained without overstating the preliminary evidence.

References

[1] Chen, B., Wang, Y., Shome, P., Fletcher, C. W., Kohlbrenner, D., Paccagnella, R., and Genkin, D. GoFetch: Breaking constant-time cryptographic implementations using data memory-dependent prefetchers. In *33rd USENIX Security Symposium* (2024).

[2] Vicarte, J. R. S., Flanders, M., Paccagnella, R., Garrett-Grossman, G., Morrison, A., Fletcher, C. W., and Kohlbrenner, D. Augury: Using data memory-dependent prefetchers to leak data at rest. In *2022 IEEE Symposium on Security and Privacy (SP)* (2022).